

What is kb, Mb, and Gb?

What is kb, Mb, and Gb?

- bit: smallest unit, only has two states, 0 and 1;
- byte: basic unit, consist 8-bits, from 00000000 to 11111111;
- Kb: The prefix 'kilo' traditionally means a thousand, like in kilogram and kilometer. However, in computer science, 1 KB (kilobyte) is not exactly 1,000 bytes but 1,024 bytes.
- Mb: The prefix 'mega' means a million. In other contexts, mega equals 10^6 . But in CS, $1\text{Mb} = 1,024\text{ kb} = 1024 * 1024\text{ bytes}$
- Gb: gigabyte. Almost equal to 10^9 , but in CS, $1\text{Gb} = 1,024\text{ Mb} = 1024 * 1024\text{ kb}$

Office hour

Dr. Jiachen Liu

2 - 4 pm

Friday, Sept. 2026 – Jan.2027

Email or Walk-in

**I may not be in my office at other time, so please
send me an email first.**



Xi'an Jiaotong-Liverpool University

西交利物浦大學

MFE113TC

Programming Foundation for Intelligent Manufacturing Engineering

Lecture 02 Fundamentals of C/C++ Programming

Dr. Jiachen Liu

jiachen.liu@xjtlu.edu.cn

Fall, 2026

Outline

- Your First C program
- C-Language Data
- Variables and Constants
- Characters and Strings
 - Characters, Arrays, and String in C
 - String in C++
- Coding Style
- Basic I/O Functions in C and C++

Your First C Program

- **Example:**

```
#include <stdio.h>

int main()
{
    printf("Programming in C is fun.\n");
    return 0;
}
```

Your First C Program

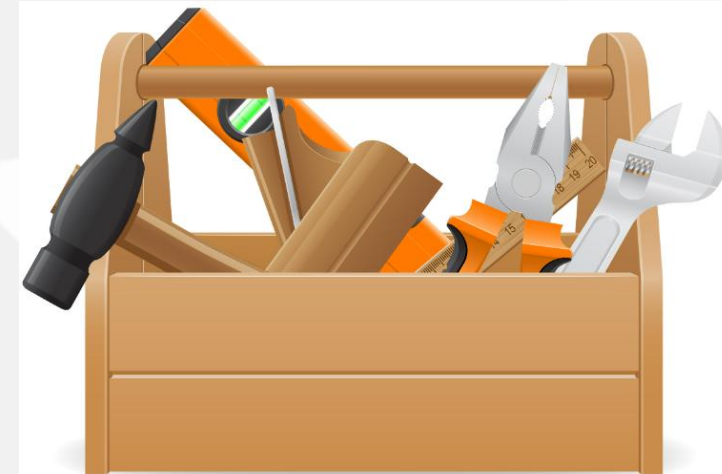
Composition of a piece of C program

- Preprocessor directives
 - Commands that give instructions to the C preprocessor, whose job is to modify the text of a program **before** it is compiled.
 - Begin with the # sign; hash key, pound sign.
 - Two commonly used directives
 - **#include**
 - **#define**

Your First C Program

The `#include` Directive

- Syntax: `#include` <STANDARD> or `#include` "USER"
- Examples:
`#include <stdio.h>`
`#include "Myheader.h"`
- Interpretations:
 - A header file is **pre-defined** by others (or ourselves); it's like a toolbox that provides useful tools for new programmers. It saves our time since **we don't have to write every function** from scratch repeatedly.



Your First C Program

The `#include` Directive

```
C test.c ×
C test.c > main()
1 #include <stdio.h>
2
3 int main(){
4     printf("Programming is fun\n");
5
6     return 0;
7 }
8
```

Go to Definition
Go to Declaration
Go to Type Definition
Go to References
Peek
Find All References
Show Call Hierarchy

```
C test.c C stdio.h ×
D: > software > mingw > mingw64 > x86_64-w64-mingw32 > include > C stdio.h > printf(const char *, ...)
333 __attribute__((__format__(gnu_printf, 2, 2))) __MINGW_ATTRIB_NONNULL(2)
334 int fprintf (FILE *__stream, const char *__format, ...)
335 {
336     int __retval;
337     __builtin_va_list __local_argv; __builtin_va_start( __local_argv, __format);
338     __retval = __mingw_vfprintf( __stream, __format, __local_argv );
339     __builtin_va_end( __local_argv );
340     return __retval;
341 }
342
343 __mingw_ovr
344 __attribute__((__format__(gnu_printf, 1, 2))) __MINGW_ATTRIB_NONNULL(1)
345 int printf (const char *__format, ...)
346 {
```

Your First C Program

The #define Directive

- Syntax: **#define** NAME value

- Examples:

#define PI 3.1415927

#define MAX 100

- Interpretations:

- The preprocessor is notified to replace each use of the identifier NAME by the defined value;

- Remarks:

- C program statements cannot change the value associated with the NAME.
- Q: What can be put in the define???

Your First C Program

The #define Directive

- Interpretations:

- The #define directive simplifies the code by replacing the specified text;

↓ What we programmed

```
#include <stdio.h>

#define PI 3.14

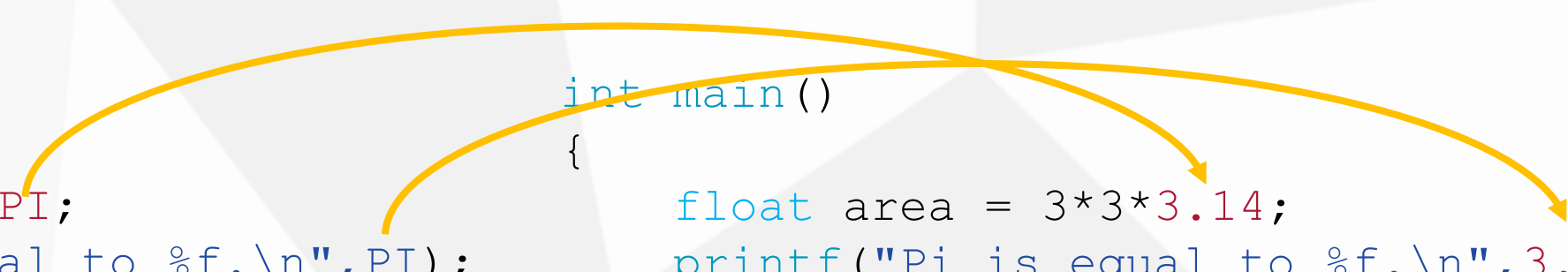
int main()
{
    float area = 3*3*PI;
    printf("Pi is equal to %f.\n",PI);
    return 0;
}
```

↓ What the compiler get

```
#include <stdio.h>

#define PI 3.14

int main()
{
    float area = 3*3*3.14;
    printf("Pi is equal to %f.\n",3.14);
    return 0;
}
```



Your First C Program

The main() Function

- Syntax:

```
int main(void)
{
    main function body
}
```

Your First C Program

The `main()` function

- There is **one and only one main function** in every C program;
- C program is **executed from the main function** -> software **entrance**;
- C program is terminated at the end of the main function -> software **exit**.
- Braces **{ }** enclose the main function body containing declarations and executable statements.

```
#include <stdio.h>
```

```
int main()  
{  
    printf("Programming in C is fun.\n");  
    return 0;  
}
```

Your First C Program

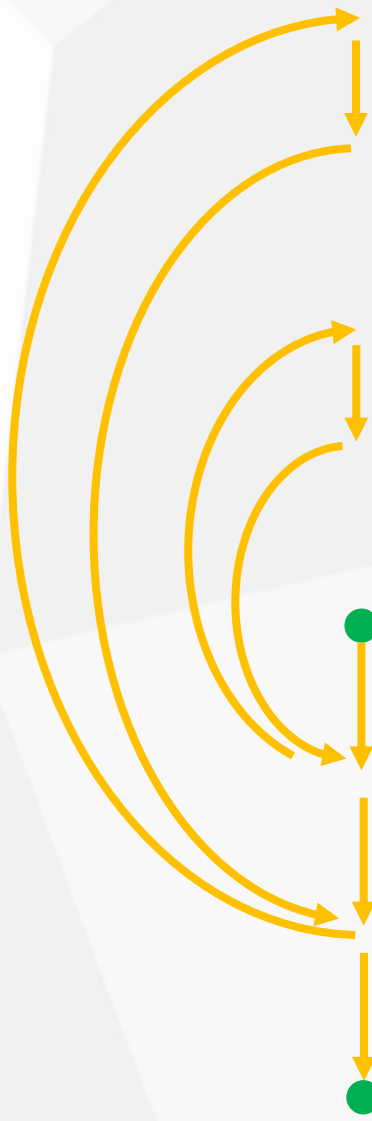
The `main()` function

```
#include <stdio.h>
```

```
void fun2 () {  
    printf("Here is function 2.\n");  
}
```

```
void fun1 () {  
    printf("Here is function 1.\n");  
}
```

```
int main () {  
    fun1 ();  
    fun2 ();  
    return 0;  
}
```



Your First C Program

Composition of a piece of C program

- The main() function
 - Observations
 - **int** main () indicates that the main function returns an **integer** value to the operating system when it finishes normal execution (the example returns zero);

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
printf("Programming in C is fun.\n");
```

```
return 0;
```

```
}
```

Your First C Program

Composition of a piece of C program

- The main() function
 - Observations
 - Why it needs 'return 0'?

```
#include <stdio.h>
```

```
int main()  
{
```

```
printf("Programming in C is fun.\n");
```

```
return 0;
```

```
}
```

Your First C Program

Composition of a piece of C program

- The main() function
 - Observations
 - Why it needs 'return 0'?
 - The engineers need **a flag** to show if the program is executed successfully or not.

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
printf("Programming in C is fun.\n");
```

```
return 0;
```

```
}
```

C-Language Data

Data Types

Examples of commonly used data types:

- **int**
- float
- double
- char
- string

46

103

-5

C-Language Data

Data Types

Examples of commonly used data types:

- int
- float
- double
- char
- string

1.06

5.00

-5.3465432

C-Language Data

Data Types

Examples of commonly used data types:

- int
- float
- double
- **char**
- string

'C'

'5'

'+'

'&'



C-Language Data

Data Types

Examples of commonly used data types:

- int
- float
- double
- char
- **string**

“Hello, World”

“RBE104TC”

“I love XJTLU”

C-Language Data

Data Types and Ranges

Remember? 1 byte consists of 8 bits, from 0000 0000 (0) to 1111 1111 (255).

Data Type	Description	Bytes	Range
short	short integer	2	-32,768 to 32,767
unsigned short	positive short integer	2	0 to 65535
int	integer	2 or 4	see short or long
unsigned int	positive integer	2 or 4	see unsigned short or long
long	long integer	4	-2,147,483,648 to 2,147,483,647
unsigned long	positive long integer	4	0 to 4,294,967,295
float	single precision real number	4	3.4×10^{-38} to 3.4×10^{38} (6–7 significant digits of precision)
double	double precision real number	8	1.7×10^{-308} to 1.7×10^{308} (15-16 digits of precision)
long double	double precision real number	12	3.4×10^{-4931} to 3.4×10^{4931} (≥ 15 -16 digits of precision)
char	character	1	-128 to 127
unsigned char	positive character	1	0 to 255

C-Language Data

Character Set in C

- The characters in C are grouped into the follow categories:
 - Letters: UPPERCASE A...Z, lowercase a...z;
 - Digits: 0...9
 - Special Characters: e.g., + = - / ! : ; etc.
 - White Space: e.g., blank space, tab, return (enter).

C-Language Data

Character Set in C

- Since computers can only understand numbers, such as a stream of binary number:

...010001000111101...

- How can a stream of binary number be converted to characters?

C-Language Data

Character Set in C

- Since computers can only understand numbers, such as a stream of binary number:

...010001000111101...

- How can a stream of binary number be converted to characters?
- Have you ever watched a spy movie?

Two spies communicated by tapping, because they were both familiar with the same **encoding** method.



C-Language Data

Character Set in C

- Game:

01000111010011110100111101000100

C-Language Data

Character Set in C

- Game:

01000111010011110100111101000100

Binary	English Char
01000001	A
01000010	B
01000011	C
01000100	D
01000101	E
01000110	F
01000111	G
01001000	H
01001001	I
01001010	J
01001011	K
01001100	L
01001101	M

Binary	English Char
01001110	N
01001111	O
01010000	P
01010001	Q
01010010	R
01010011	S
01010100	T
01010101	U
01010110	V
01010111	W
01011000	X
01011001	Y
01011010	Z

C-Language Data

ASCII

- ASCII (in total $2^7=128$)
- ASCII is short for The **A**merican **S**tandard **C**ode for **I**nformation **I**nterchange;
- It is a standard **7**-bit code proposed by ANSI (**A**merican **N**ational **S**tandard **I**nstitute) in 1963, and finalised in 1968;
- Since computers can only understand numbers, so an ASCII code is the numerical representation of a character such as **a** or **@** (coded as **97** and **64** respectively).
 - Extended ASCII = 8 bits = 256 possible values
 - 0–127: Standard ASCII
 - 128–255: extra extendable character (but this maybe different by different systems)

C-Language Data ASCII Table

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]	48	30	110000	60	0	96	60	1100000	140	`
1	1	1	1	[START OF HEADING]	49	31	110001	61	1	97	61	1100001	141	a
2	2	10	2	[START OF TEXT]	50	32	110010	62	2	98	62	1100010	142	b
3	3	11	3	[END OF TEXT]	51	33	110011	63	3	99	63	1100011	143	c
4	4	100	4	[END OF TRANSMISSION]	52	34	110100	64	4	100	64	1100100	144	d
5	5	101	5	[ENQUIRY]	53	35	110101	65	5	101	65	1100101	145	e
6	6	110	6	[ACKNOWLEDGE]	54	36	110110	66	6	102	66	1100110	146	f
7	7	111	7	[BELL]	55	37	110111	67	7	103	67	1100111	147	g
8	8	1000	10	[BACKSPACE]	56	38	111000	70	8	104	68	1101000	150	h
9	9	1001	11	[HORIZONTAL TAB]	57	39	111001	71	9	105	69	1101001	151	i
10	A	1010	12	[LINE FEED]	58	3A	111010	72	:	106	6A	1101010	152	j
11	B	1011	13	[VERTICAL TAB]	59	3B	111011	73	;	107	6B	1101011	153	k
12	C	1100	14	[FORM FEED]	60	3C	111100	74	<	108	6C	1101100	154	l
13	D	1101	15	[CARRIAGE RETURN]	61	3D	111101	75	=	109	6D	1101101	155	m
14	E	1110	16	[SHIFT OUT]	62	3E	111110	76	>	110	6E	1101110	156	n
15	F	1111	17	[SHIFT IN]	63	3F	111111	77	?	111	6F	1101111	157	o
16	10	10000	20	[DATA LINK ESCAPE]	64	40	1000000	100	@	112	70	1110000	160	p
17	11	10001	21	[DEVICE CONTROL 1]	65	41	1000001	101	A	113	71	1110001	161	q
18	12	10010	22	[DEVICE CONTROL 2]	66	42	1000010	102	B	114	72	1110010	162	r
19	13	10011	23	[DEVICE CONTROL 3]	67	43	1000011	103	C	115	73	1110011	163	s
20	14	10100	24	[DEVICE CONTROL 4]	68	44	1000100	104	D	116	74	1110100	164	t
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]	69	45	1000101	105	E	117	75	1110101	165	u
22	16	10110	26	[SYNCHRONOUS IDLE]	70	46	1000110	106	F	118	76	1110110	166	v
23	17	10111	27	[ENG OF TRANS. BLOCK]	71	47	1000111	107	G	119	77	1110111	167	w
24	18	11000	30	[CANCEL]	72	48	1001000	110	H	120	78	1111000	170	x
25	19	11001	31	[END OF MEDIUM]	73	49	1001001	111	I	121	79	1111001	171	y
26	1A	11010	32	[SUBSTITUTE]	74	4A	1001010	112	J	122	7A	1111010	172	z
27	1B	11011	33	[ESCAPE]	75	4B	1001011	113	K	123	7B	1111011	173	{
28	1C	11100	34	[FILE SEPARATOR]	76	4C	1001100	114	L	124	7C	1111100	174	
29	1D	11101	35	[GROUP SEPARATOR]	77	4D	1001101	115	M	125	7D	1111101	175	}
30	1E	11110	36	[RECORD SEPARATOR]	78	4E	1001110	116	N	126	7E	1111110	176	~
31	1F	11111	37	[UNIT SEPARATOR]	79	4F	1001111	117	O	127	7F	1111111	177	[DEL]
32	20	100000	40	[SPACE]	80	50	1010000	120	P					
33	21	100001	41	!	81	51	1010001	121	Q					
34	22	100010	42	"	82	52	1010010	122	R					
35	23	100011	43	#	83	53	1010011	123	S					
36	24	100100	44	\$	84	54	1010100	124	T					
37	25	100101	45	%	85	55	1010101	125	U					
38	26	100110	46	&	86	56	1010110	126	V					
39	27	100111	47	'	87	57	1010111	127	W					
40	28	101000	50	(	88	58	1011000	130	X					
41	29	101001	51	)	89	59	1011001	131	Y					
42	2A	101010	52	*	90	5A	1011010	132	Z					
43	2B	101011	53	+	91	5B	1011011	133	[					
44	2C	101100	54	,	92	5C	1011100	134	\					
45	2D	101101	55	-	93	5D	1011101	135	]					
46	2E	101110	56	.	94	5E	1011110	136	^					
47	2F	101111	57	/	95	5F	1011111	137	_					

C-Language Data

ASCII Table

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
48	30	110000	60	0	96	60	1100000	140	`
49	31	110001	61	1	97	61	1100001	141	a
50	32	110010	62	2	98	62	1100010	142	b
51	33	110011	63	3	99	63	1100011	143	c
52	34	110100	64	4	100	64	1100100	144	d
53	35	110101	65	5	101	65	1100101	145	e
54	36	110110	66	6	102	66	1100110	146	f
55	37	110111	67	7	103	67	1100111	147	g
56	38	111000	70	8	104	68	1101000	150	h

- The following relationship will hold:
 - '0' < '1' < '2' < ... < '9' (48-57);
 - 'A' < 'B' < 'C' < ... < 'Z' (65-90);
 - 'a' < 'b' < 'c' < ... < 'z' (97-122).

C-Language Data

ASCII Table

- Example:

```
#include <stdio.h>
```

```
int main()  
{
```

```
    char t = '0';
```

```
    printf("The ASCII value of 0 is:%d\n", t);
```

```
    t = 'a';
```

```
    printf("The ASCII value of a is:%d\n", t);
```

```
    t = '@';
```

```
    printf("The ASCII value of @ is:%d\n", t);
```

```
    t = ' ';
```

```
    printf("The ASCII value of ' ' is:%d\n", t);
```

```
    return 0;
```

```
}
```

%d means output an integer

%.2f means output a float number
to 2 decimal places

\n: enter next line

C-Language Data

Symbols used in programming

- **;** represents a statement; <- Semicolon
- **,** represents the separator of expression; <- comma
- **:** represents switch branches; <- colon
- **{ }** represents block mark; <- brace bracket
- **()** is used in the function parameter or expression grouping; <- bracket
- **[]** defines array or index the elements of arrays; <- square bracket
- **%** represents the formatter or solving reminder; <- percentage
- ...

Try it by yourself !

C-Language Data

Keywords in C

- Some keywords are fixed meanings, which cannot be changed;
- They **MUST** be written in **lowercase**;
- They are used as basic building blocks;
- There are roughly 32 keywords in C programming;
- Some examples:

break	case	char
const	do	else
float	for	if
int	return	sizeof
static	struct	typedef
union	void	while

C-Language Data

Standard Library Functions in C

- These functions are given special meaning;
- They can be redefined by the programmer
 - But it is **NOT recommended!**
- Examples:

Standard Libraries	Description	Functions
stdio.h	Standard I/O Library for C	printf, scanf, ...
iostream	Standard I/O Library for C++	cin, cout, ...
math.h	Standard mathematic library	sin, cos, tan, log, ...
time.h	Standard time library	time, ...
...	...	...

C-Language Data

User-defined identifiers

- These identifiers refer to names of **variables, functions, and arrays**.
- They are defined by the user / programmer (That's **YOU**).
- Names can be chosen following these rules:
 - First character **must** be a **letter** or **underscore** (**_**);
 - Distinguish between - (**hyphen / subtraction**) and **_** (**underscore**);
 - **hyphen should not be used; underscore can be used;**
 - **Must** only contain characters, digits, underscores;
 - Must **not** contain whitespace;
 - Avoid excessively long identifiers (better ≤ 31).
 - (**Suggestion**) Should be with a **descriptive name**.
- Q: Why we want a descriptive name?

C-Language Data

User-defined identifiers

- Are the following examples valid identifiers?
 - Hello_My_Name_Is_Tom
 - hello_my_name_is_tom
 - F12345
 - 1F2345
 - F12 45

C-Language Data

User-defined identifiers

- Are the following examples valid identifiers?
 - Hello_My_Name_Is_Tom
 - hello_my_name_is_tom
 - F12345
 - 1F2345  Start from a digit
 - F12 45  Contain space

C-Language Data

User-defined identifiers

- In terms of a good habit from scratch, which one of the following is a good name for a variable? (Hint: with a **descriptive name**)
- Possible candidates:
 - `student_avg_score` (for function name or variable name);
 - `_student_avg_score` (for local variables);
 - Camel Usage:
 - `StudentAvgScore` (for class or struct definition);
 - `studentAvgScore` (for variable).

Variables and Constants

Variables and Variable Declarations

- **A variable:**
 - is a data name that can be used to store a data value;
 - may take **different values at different times** during program execution;
 - **associated with a data type** (i.e., a character, or a number).
- **Variable declarations:**
 - tells the compiler the variable name;
 - specifies the type of data the variable will hold.
- **Syntax:** **data_type** name
 - Examples: **int** number, **char** name, **double** sum, total.

Variables and Constants

Why use variables (a story)

```
#include <stdio.h>

int main()
{
    printf("1 + 2 = 3\n");
    printf("1 * 2 = 2\n");
    printf("2 * 1 + 3 * 2 = 8\n");
    return 0;
}
```

Variables and Constants

Why use variables (a story)

```
#include <stdio.h>

int main()
{
    printf("3 + 4 = 7\n");
    printf("3 * 4 = 12\n");
    printf("2 * 3 + 3 * 4 = 18\n");
    return 0;
}
```

Variables and Constants

Why use variables (a story)

- Variable is a “nickname” for a data that will be used in multiple times.

```
#include <stdio.h>

int main()
{
    int a = 1;
    int b = 2;
    printf("%d + %d = %d\n",a,b,a+b);
    printf("%d * %d = %d\n",a,b,a*b);
    printf("2 * %d + 3 * %d = %d\n",a,b,2*a+3*b);
    return 0;
}
```

Variables and Constants

Why use variables (a story)

- Variable is a “nickname” for a data that will be used in multiple times.

```
#include <stdio.h>

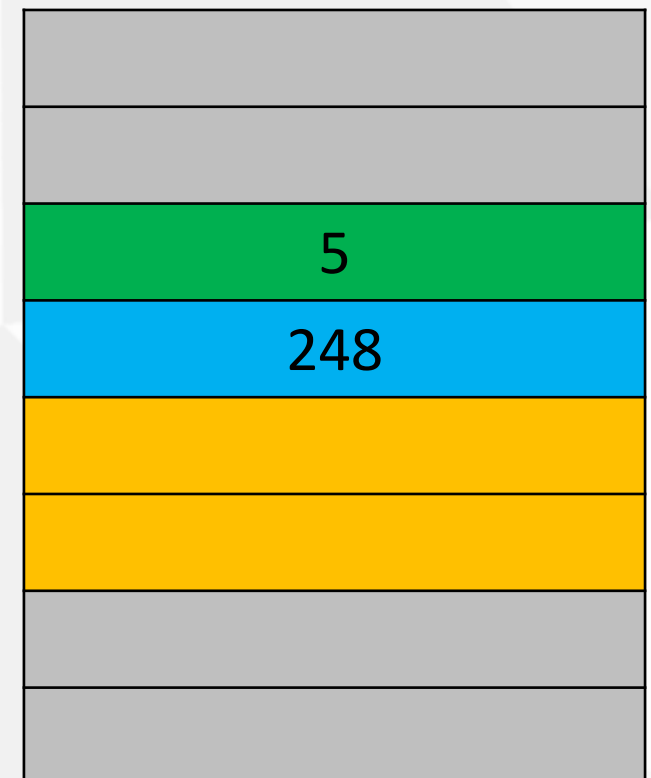
int main()
{
    int a = 3;
    int b = 4;
    printf("%d + %d = %d\n",a,b,a+b);
    printf("%d * %d = %d\n",a,b,a*b);
    printf("2 * %d + 3 * %d = %d\n",a,b,2*a+3*b);
    return 0;
}
```

Variables and Constants

Variables

- A variable is a **named memory location** that holds a value (in binary);
- in C (and most languages), **a variable must be declared before use**;
- Declaration is in charge of TWO things:
 - Determines the **type** of variable (how we can use it);
 - **Reserves specific memory space** for the variable.

```
int a=5;  
int b=248;  
  
float c;
```



Variables and Constants

Constants

- Refer to fixed values that cannot be changed during the execution of a C program;
 - Using the keyword **const** before the variable declaration prevents the value from being changed. E.g., :
 - **const int** a=7;
 - **const float** pi=3.141;
 - **const char** msg[]="warning";
- What's the difference between **#define** and **const**?

Variables and Constants

Constants

- Refer to fixed values that cannot be changed during the execution of a C program;
 - Using the keyword **const** before the variable declaration prevents the value from being changed. E.g., :
 - **const int** a=7;
 - **const float** pi=3.141;
- What's the difference between **#define** and **const**?
 - More specific: const is a keyword that declares a constant with a **specific data type**.
 - More flexible: #define creates identifiers that are **global to the entire program** (or file), while const can have a narrower scope, such as local to a **function**.

Variables and Constants

Enumeration Constant

- Another kind of constant called enumeration constant, for example:
enum boolean {No, Yes};
 - It is equivalent to assigning 0 to No, and 1 to Yes;
- **The first name in an enum is valued as 0, and the next is valued 1, then 2,3,4,.....;** enum color {Red, Green, Blue};
- Unless explicit values are specified, e.g.,
 - enum months {JAN=1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC};
 - In this case, JAN=1, FEB=2, MAR=3, ...

Variables and Constants

Variable Initialization

```
#include <stdio.h>
int main()
{
    int a = 3; /*declare and initialize the variable*/

    /*declare the variable first, assign value to it later*/
    float x;
    x = 3.2;

    char name = 'M'; /*quotation*/
    return 0;
}
```

Characters, **Arrays**, and Strings in C

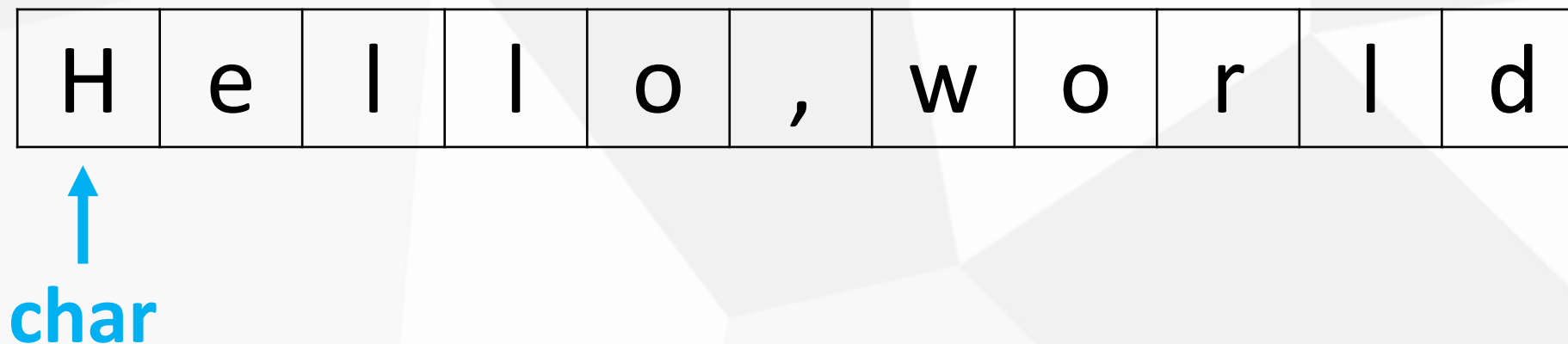
Characters and Arrays (Definition)

- Characters:
 - Characters in C consist of any printable or non-printable characters in the computer character set including **lowercase letters, decimal digits, special characters, and escape sequences**;
 - A character is usually stored in the computer as **an 8-bit (1 byte) integer**;
 - E.g., 'A', '2', '#', ...
- If **char** can only store 1 character, then why C/C++ can print a whole sentence like **"Hello, world"**?

Characters, **Arrays**, and Strings in C

Characters and Arrays

- Arrays:
 - **Ordered sequences of data elements** of the same type;
 - In simple works, several memory cells in a row.
 - E.g., {'A', 'B', 'C', 'D', 'E', 'F', 'G'}.



Characters, **Arrays**, and Strings in C

Defining an array

- How to declare an array: **char** name[20];
- This statement declares an array named **name** with **20** elements (i.e., name can store 20 characters);
- The square brackets indicate an array;
- An array **MUST** be assigned a dimension (the number of elements) **except the initialization of the array**;
- E.g.,
int attendance[15];
int scores[]={1,2,3};
float daily_temperature[30];
char name[]="Mike";

Characters, **Arrays**, and Strings in C

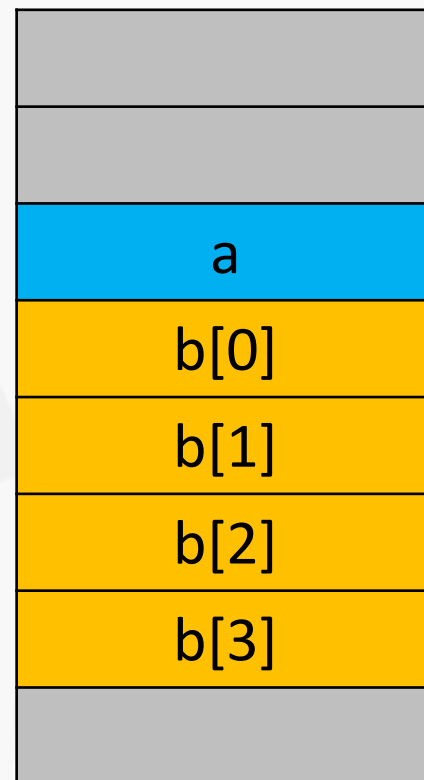
Accessing array elements

- Declarations:

```
int a;
```

```
int b[4];
```

```
/* declare an array  
with 4 ints */
```



- Assignment:

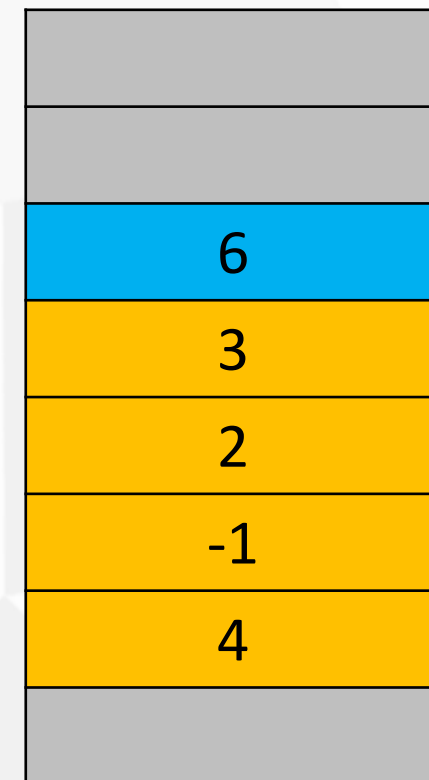
```
a = 6;
```

```
b[0] = 3;
```

```
b[1] = 2;
```

```
b[2] = -1;
```

```
b[3] = 4;
```



Characters, **Arrays**, and Strings in C

Accessing array elements

- How can I use each element of my array?
- The first element is always indexed as 0;
- **DO NOT EXCEED array bounds !**
- Consider the following:

```
float price[5]; /* declare an array with 5 elements */
```

```
price[0]=12.12; /* assign the value to the first element */
```

```
price[1]=13.13; /* assign the value to the second element */
```

```
...
```

Characters, **Arrays**, and Strings in C

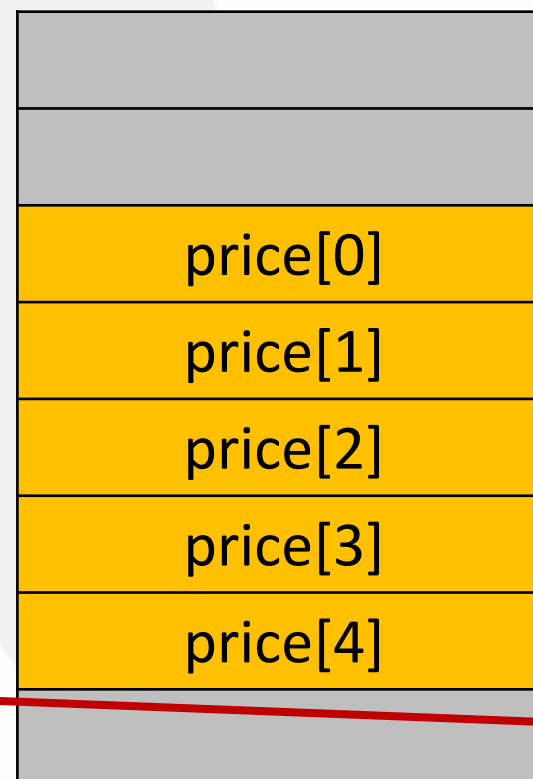
Accessing array elements

`price[5]=12.34;` */* assign the value to the last+1 element */*

Nothing wrong will be prompted, but it is strongly recommended not to do so...

```
#include <stdio.h>
```

```
int main(void)
{
    float price[5];
    price[0]=12.12;
    price[1]=13.13;
    price[5]=12.34;
}
```



Characters, **Arrays**, and Strings in C

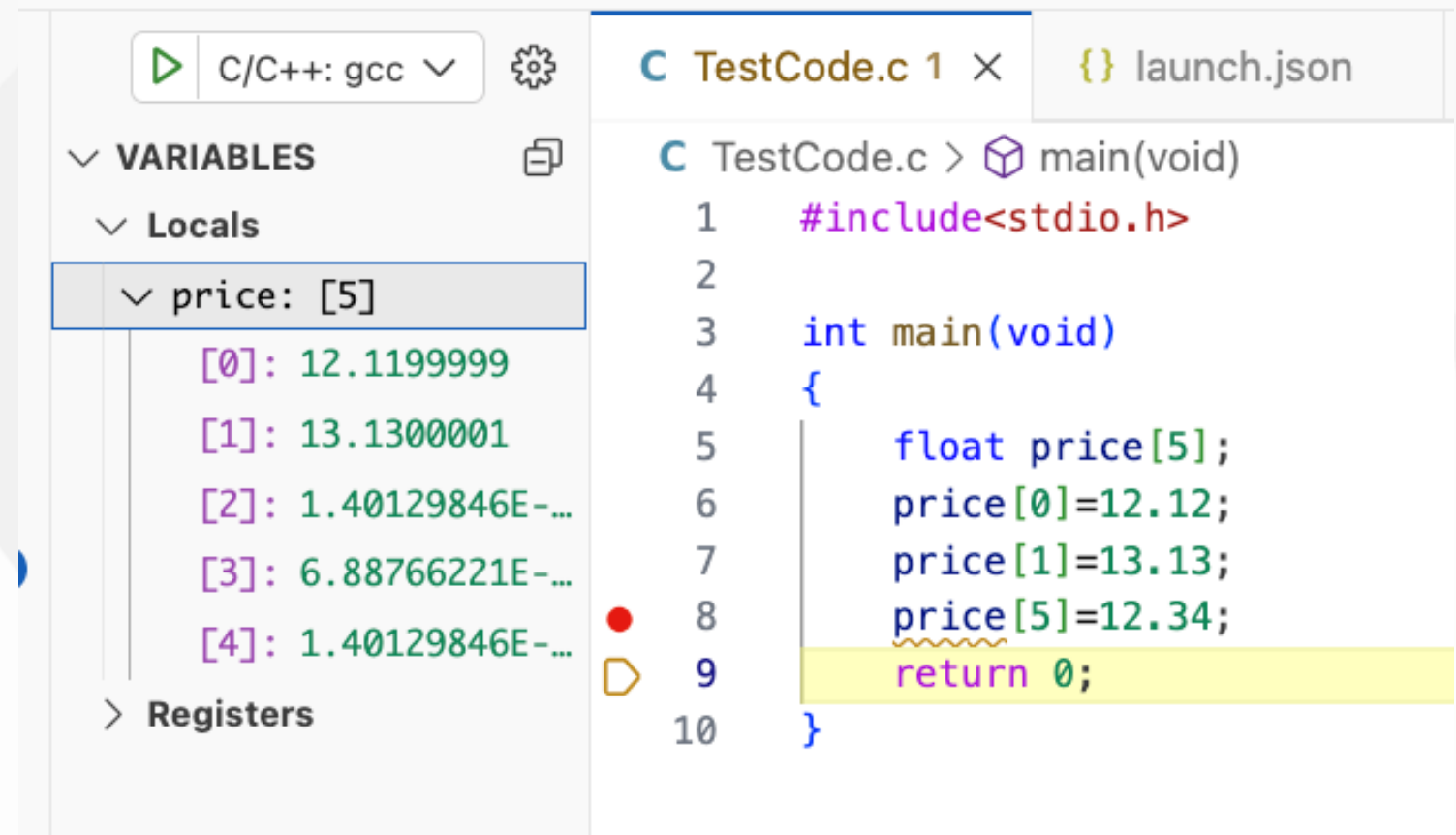
Accessing array elements

`price[5]=12.34; /* assign the value to the last+1 element */`

Nothing wrong will be prompt, but it is strongly recommended not to do
so...

```
#include <stdio.h>

int main(void)
{
    float price[5];
    price[0]=12.12;
    price[1]=13.13;
    price[5]=12.34;
}
```



Characters, **Arrays**, and Strings in C

Defining an array

- Two way to define an array:

```
int scores[3]={1,2,3};
```

- OR:

```
int scores[3];  
scores[0] = 1;  
scores[1] = 2;  
scores[2] = 3;
```

- An array **MUST** be assigned a dimension (the number of elements) **except the initialisation of the array**;
- E.g.,

```
int scores[]={1,2,3};  
char name[]="Mike";
```



```
int scores[  
scores[0] = 1;
```



Characters, Arrays, and **Strings** in C

String Fundamentals

- A string is an **array of characters**.
- However, string should **end with the NULL character or '\0'**;
- It can be written with double quotes "I am a string";
- It can be assigned when being declared:

```
char firstname[] = "Walter";  
char lastname[] = "White";  
char fullname[] = "Walter White";
```
- #define can be used to create a string:

```
#define TV "Breaking Bad"
```

Characters, Arrays, and **Strings** in C

String Fundamentals

- A string can also be defined by specifying individual characters:
 - **char** color[] = {'g','r','e','e','n','**\0**'};
 - It specifies that each character is stored in its own element;
 - Print for each individual character:

```
printf("The second character is %c", color[1]);
```

Characters, Arrays, and **Strings** in C

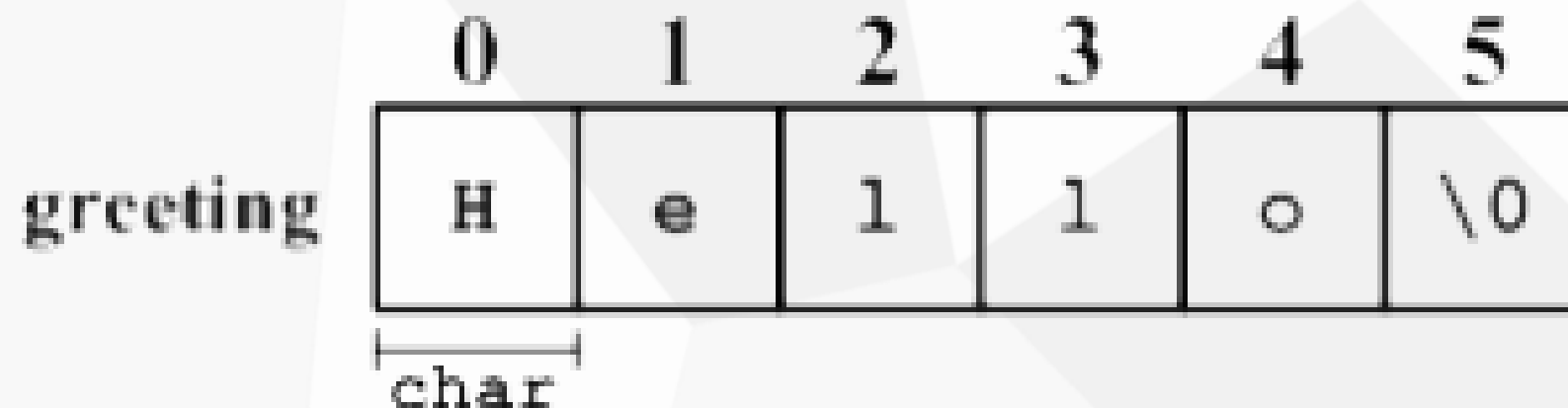
String Library Functions

- There is a library of string functions ***string.h***;
 - ***#include<string.h>***
- Some Operation Examples:
 - `strlen(str)`: finds the length of `str`;
 - `strcmp(str1, str2)`: compares two strings, `str1` and `str2`, character by character;
 - `strcpy(str1, str2)`: copies a string from one array (`str1`) to another array (`str2`);
- Try to use these functions during your lab sessions !

String in C

C-style string

- C-style string: to treat the character sequence as a **char** type array, **terminated with a null character ('\\0')**.
 - E.g.:
 - **char** greeting[5] = {'H','e','l','l','o'}; // Just an array of chars;
 - **char** greeting[6] = {'H','e','l','l','o','\\0'}; // Can be treated as a string.
- The C-style string (char array) is represented in the memory system such as:



String in C

C-style string

- In C, strings are not a distinct data type but are represented as arrays of characters. The standard way to declare a string in C is by using character arrays:

```
char str[] = "This is a string";
```

- Why we can not directly treat string as new data_type and initialize it by string?

String in C++

string is in the std namespace. Use std::string, or write using namespace std; first and then use string.

C++-style string

- The C++ standard expanded the C library by introducing a **string** class, the alternative way to deal with character sequence:
 - To use the **string** class, a program has to include the **<string>** header file;
 - The string class is part of the **std** namespace, so it is necessary to provide a using directive: **using namespace std**.
 - #include<string>
 - using namespace std;
- To declare and initialize a string object, it is similar to declare and initialize an ordinary variable, e.g.:

```
string str1;           // declare an empty string object
string str2 = "Hello"; // initialise with the assignment style
string str3("World");  // initialise with the constructor style
```

String in C++

string is in the std namespace. Use std::string, or write using namespace std; first and then use string.

C++-style string

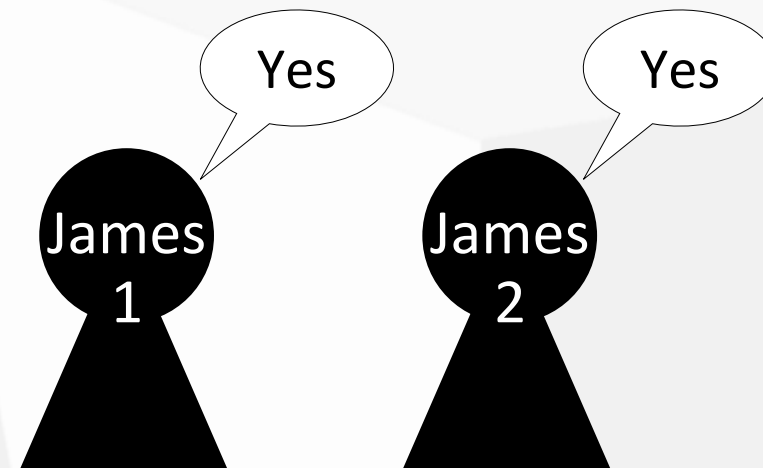
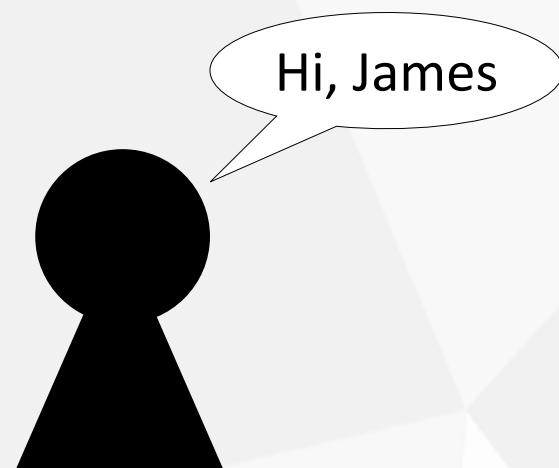
- C++ provides the string class for working with text.
- To use string, include the <string> header:
`#include <string>`
- string belongs to the std namespace. You can write:
`std::string name;`
- Or, after writing: `using namespace std;` you can simply write: `string name;`
- To declare and initialize a string object, it is similar to declare and initialize an ordinary variable, e.g.:

<code>string str1;</code>	<code>// declare an empty string object</code>
<code>string str2 = "Hello";</code>	<code>// initialize with the assignment style</code>
<code>string str3("World");</code>	<code>// initialize with the constructor style</code>

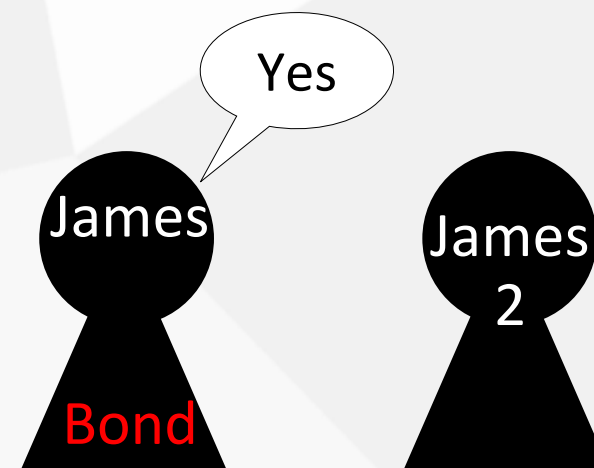
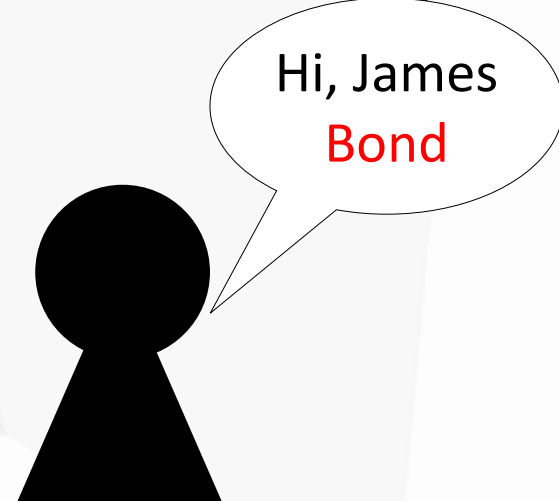
String in C++

C++-style string

- What is a **namespace**? Imagine there are two students named James in the same class. How can we specify which James we want to call?



- We can call them by their full names.



String in C++

C++-style string

- Similarly, in a large programming project, there would be many engineers working together. It's almost inevitable that sometimes two engineers might create **functions with the same name**.
- Therefore, **namespaces** are needed to further distinguish these functions. For instance, the namespace of standard header files is **std**.
- For example:

std::cout, std::cin, std::string, std::endl, etc.
cout, cin, string, endl, etc.

Coding Styles

Comment Lines

- Comment lines are for you to **give descriptive information** about your codes.
- The C compiler will ignore comment lines.
- Typical C style comments:

```
/* This is a C style comment */
```

```
/******
```

This is also acceptable

```
*****/
```

- C++ style comments (Both types of comments **can now be mixed and used in C/C++**, but they still need to be used with care):

```
// It is C++ style comments.
```

Coding Styles

Comment Lines

- You can even use comment lines to write a joke for users.

```
#include <stdio.h>
int main()
{
    /*No! I'm not happy, It's Wednesday,
    I still have to work for 2 days.*/

    printf("I am so happy today.");

    return 0;
}
```

Coding Styles

Statements

- The body of the main function is a set of statements;
- Statements are instructions to the computer that you want the computer is going to do for you;
- The end of a statement is a **semicolon (;)**
- Blank space will be ignored;
- Statements are **executed in sequence** from the top to the bottom.

Coding Styles

Statements

```
#include <stdio.h>
int main( )
{
    printf("Hello. \n"); /*Statement 1*/
    printf("How are you. In"); /*Statement 2*/
    printf("I am fine, thanks. And you? \n"); /*Statement 3*/
    printf("I am fine too, thanks. \n"); /*Statement 4*/
    return 0;
}
```

Coding Styles

Assignment Statements

- Syntax: variable = expression;
- The equal sign (=) is called the assignment operator;
- Expression is evaluated first then assigned to variables.

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int a; /*declarations*/  
    float x;  
    a=3; /*assignments: a is assigned to value 3*/  
    x=a+3.2; /* guess what should be the value of x?? */  
    return 0;  
}
```

Coding Styles

Math Statements

- Math expressions:

E.g., a , $a + b$, $a \% 10$, $a + c / d$, etc.;

- In general, the math expression can only occur on the **right side** of the assignment:

E.g., $c = a + d / e$, etc.;

- Operators are assigned different priorities during the computation:

E.g., The priority of $*$ $/$ is higher than $+$ $-$.

Similar to what we learn in primary school

Basic I/O Functions

Basic I/O Functions for Characters and Strings in C Coding

- Standard input functions:
 - getchar() (for characters)
 - **scanf() (for various types of input, include characters and strings)**
 - gets() (for strings)
- Standard output functions:
 - putchar() (for characters)
 - **printf() (for various types of output, include characters and strings)**
 - puts() (for strings)

Basic I/O Functions

The Function `printf()`

- It is used to output (usually to the screen);
 - Without argument:
`printf("hello");` —> it just prints the letters hello;
 - With argument (expressions to evaluate):
the following common format **specifiers** are used:

Type	Specifier
int	%d
float	%f
double	%lf (long float)
char	%c
string	%s

Basic I/O Functions

The Function `printf()`

```
#include <stdio.h>
int main()
{
    int x = 72;
    char b = 'Z';
    float c = 3.141;
    printf("x equals %d\n", x); /* On the screen: x equals 72*/
    printf("b equals %c\n", b); /* On the screen: b equals Z*/
    printf("%d multiplied by %f equals %f\n", x, c, x*c);
    /* On the screen: 72 multiplied by 3.141 equals 226.152*/
    return 0;
}
```

Basic I/O Functions

The Function `printf()`

- Left / Right Alignment and Digits after the **Decimal Point**.
 - if the actual digits before the decimal point is smaller than m
 - “%m.nf” means:
 - **m: minimum positions it will take up**
 - **n: positions after decimal point**
 - **-: left alignment**
 - **no - : right alignment**
 - if the actual digits before the decimal point is greater than m, keep the original digits.

```
#include<stdio.h>

int main()
{
    float a=3.14159265;
    printf("a=%f\n",a);
    printf("a=%6.4f\n",a);
    printf("a=%2.3f\n",a);
    printf("a=%9.5f\n",a);
    printf("a=%-9.5f\n",a);
    printf("a=%5.0f\n",a);
    return 0;
}
```

a=3.141593
a=3.1416
a=3.142
a= 3.14159
a=3.14159
a= 3

Space

Basic I/O Functions

The Function `scanf()`

- It **reads data from the standard input device `stdin`** (usually the keyboard) and starts the read information in a variable;
- General syntax: `scanf("format specifier", &variable);`
- The **ampersand (&)**: specifies the **memory address** of the variable;

Details will be given in future
lectures



Basic I/O Functions

The Function `scanf()`

- The same common format specifier used in `printf()` are used in `scanf()`:

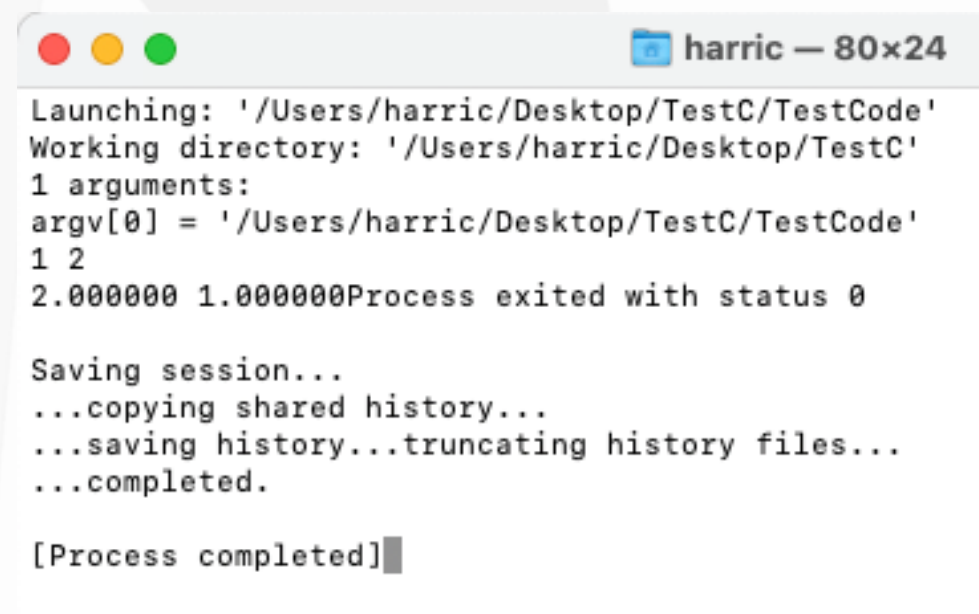
Type	Specifier
int	%d
float	%f
double	%lf (long float)
char	%c
string	%s

Basic I/O Functions

The Function `scanf()`

- We can add more specifiers to enter more than one argument:

```
#include <stdio.h>
int main()
{
    float height, weight;
    scanf("%f %f", &height, &weight);
    printf("%f %f", height+1, weight-1);
    return 0;
}
```



```
harric — 80x24
Launching: '/Users/harric/Desktop/TestC/TestCode'
Working directory: '/Users/harric/Desktop/TestC'
1 arguments:
argv[0] = '/Users/harric/Desktop/TestC/TestCode'
1 2
2.000000 1.000000
Process exited with status 0

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Process completed]
```

C++ Basic Input and Output

Standard Input: `cin`

- The standard input device is usually the keyboard;
- Handling the standard input in C++ is done by applying the overloaded operator of extraction (`>>`, **angle bracket**) on the `cin` stream;
- The operator **must be followed** by the variable that will store the data **that is going to be extracted from the stream**; e.g.:

```
int age;
```

```
cin >> age;
```

- `cin` can be used to request **more than one** input from the keyboard:

```
cin >> a >> b;
```

- `cin` can also be used to get strings:

```
cin >> mySurname;
```

```
getline(cin, myFullName);
```

once it reads space, input ends. So, Eddie Liu
can read the full name including space

C++ Basic Input and Output

Standard Output: **cout**

- By default, the standard output of a program is the screen (monitor), and the C++ stream object defined to access it is **cout**;
- **cout** is used in connection with the insertion operator, which is written as `<<` , e.g.:

```
cout << "Output sentence"; // prints content in " " on the screen
```

```
cout << 120; // prints number 120 on the screen
```

```
cout << x; // prints content of x on the screen
```

- **cout** does not add a line break after the output:

```
cout << "Hello";
```

```
cout << "Hello\n";
```

```
cout << "Hello" << endl;
```

} These two will
break the line

- The insertion operator `<<` may be used **more than once** in one statement:

```
cout << "Hello, I am " << age << "years old." << endl;
```

C++ Basic Input and Output

Example 1: `std::cin` & `std::cout`

```
#include <iostream>
#include <string>

int main() {
    int a;
    std::cin >> a;
    std::cout << "a's value is:" << a << std::endl;

    std::string str;
    std::cin >> str;
    std::cout << str << std::endl;
    return 0;
}
```

`g++ filename.cpp -o filename.exe`
`.\filename.exe`

After running, first insert a value. e.g., 10
Then insert a 'hello', see the output

C++ Basic Input and Output

Example 2: **cin** & **cout**

```
#include <iostream>
#include <string>
```

```
using namespace std;
```

← If use this, you don't need to write std:: again; std is a standard library

```
int main() {
    int a;
    cin >> a;
    cout << "a's value is:" << a << endl;

    string str;
    cin >> str;
    cout << str << endl;

    return 0;
}
```

Thanks !